

Stock Price Forecasting:

Coca-Cola (KO) Closing Price Prediction

Point and Probabilistic Forecast

With and Without Sentiments

Using Statistical, Machine Learning and Deep Learning Models

Applied Forecasting Methods

Submitted By:

Het Katrodiya	202518005
Divya Mashruvala	202518042
Aksh Patel	202518046
Sanil Shah	202518060

Instructor: Prof. Pritam Anand

Institution: Dhirubhai Ambani University

Abstract

This report presents a comprehensive forecasting study on the daily closing price of Coca-Cola (KO) stock, using data spanning from October 2009 to early 2024. The primary objective is to forecast future closing prices through a progressive modelling pipeline that begins with classical statistical time series methods and extends to advanced deep learning and probabilistic forecasting frameworks.

The dataset comprises daily OHLCV (Open, High, Low, Close, Volume) observations enriched with externally derived sentiment signals generated using a GPT-based natural language processing tool. After systematic preprocessing and stationarity analysis, the target series was confirmed to be non-stationary in its raw form, necessitating first differencing for subsequent statistical modelling.

The modelling pipeline encompasses baseline approaches (Naive Forecast, 7-Day Moving Average), autoregressive models (AR(1), AR(5)), and ARIMA-family models with and without exogenous variables (ARIMAX) across multiple feature engineering configurations. Deep learning models—Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Gated Recurrent Unit (GRU)—were implemented in both sentiment-free and sentiment-augmented variants. Finally, probabilistic interval forecasting was conducted using Gradient Boosting Regressor (GBR) and LightGBM with quantile regression.

Key findings indicate that while simple AR and Naive models achieve high R^2 scores due to price persistence, the GRU with a 120-day lookback window and sentiment features demonstrated the strongest interval forecasting performance, achieving approximately 93–94% coverage with the tightest prediction intervals. LightGBM similarly provided reliable uncertainty quantification, making it well-suited for risk-aware financial forecasting applications.

Introduction

Background and Motivation

Financial markets generate vast quantities of sequential data, and the prediction of asset prices has long been a central challenge in quantitative finance, econometrics, and machine learning research. Stock price forecasting holds significant practical value: accurate predictions inform investment strategies, portfolio risk management, derivative pricing, and algorithmic trading decisions.

The Coca-Cola Company (ticker: KO) is a globally recognised, large-capitalisation equity whose price behaviour reflects both macroeconomic conditions and consumer sentiment dynamics. As a defensive consumer staple stock, KO exhibits a relatively smoother price trajectory compared to technology or growth equities; however, it remains subject to sudden market dislocations, news-driven sentiment shifts, and broader market systemic risks—as evidenced by the COVID-19-induced crash in early 2020 clearly visible in the historical price data.

Traditional time series models such as ARIMA have been widely employed for financial forecasting, yet they are constrained by linearity assumptions and their inability to incorporate non-historical exogenous information. The rapid advancement of deep learning architectures—particularly recurrent models such as RNN, LSTM, and GRU—offers a compelling alternative capable of learning complex, non-linear temporal dependencies directly from data. Furthermore, the emergence of large language models enables the systematic extraction of market sentiment from financial news, providing an additional informational signal that may improve forecast quality.

This project is motivated by the need to evaluate and compare these modelling paradigms comprehensively on a real-world financial time series, assessing both point forecasting accuracy and interval-based uncertainty quantification.

Problem Statement

The core forecasting task is defined as follows: given a historical sequence of daily observations of the Coca-Cola (KO) stock, predict the next-day closing price. The problem is extended in two dimensions. First, the project investigates the incremental value of incorporating exogenous variables—specifically trading volume and market sentiment derived from financial news—beyond the univariate price series. Second, the project moves beyond point forecasting to construct calibrated prediction intervals, acknowledging that financial decision-making requires an assessment of forecast uncertainty rather than a single deterministic value.

Objectives

The primary objectives of this project are as follows:

1. To perform rigorous exploratory data analysis and stationarity testing on the KO closing price series.
2. To establish baseline forecasting performance using Naive and Moving Average models.
3. To implement and evaluate ARIMA-family models, systematically exploring the role of exogenous features and feature engineering.
4. To implement deep learning sequence models (RNN, LSTM, GRU) and assess the marginal contribution of sentiment information.
5. To construct calibrated prediction intervals using probabilistic forecasting models, enabling uncertainty-aware financial decision support.
6. To compare all models using a consistent evaluation framework and identify the most appropriate modelling strategy for this dataset.

Introduction To Data

Data Source

The dataset comprises daily market data for the Coca-Cola Company (NYSE: KO), covering the period from October 2009 to early 2024. The historical price data was obtained from a financial data source [Insert data source here, e.g. Yahoo Finance]. Sentiment annotations were generated externally using a GPT-based language model applied to financial news articles pertaining to the Coca-Cola stock.

Data Characteristics

The dataset is a time-indexed daily frequency panel. Each observation corresponds to one trading day and contains the following variables:

- **Date:** Timestamp serving as the temporal index.
- **Open, High, Low:** Intraday price extremes and opening price.
- **Close:** Daily closing price — the target variable for all forecasting models.
- **Adj Close:** Split and dividend-adjusted closing price.
- **Volume:** Daily trading volume in number of shares.
- **Sentiment_gpt:** A discrete sentiment score (e.g., 2.0 or 3.0) derived from GPT-based analysis of financial news.
- **News_flag:** A binary indicator (0 or 1) denoting whether a relevant news article was published on the corresponding trading day.
- **Scaled_sentiment:** A normalised continuous representation of the sentiment score.

The dataset contains approximately 3,554 daily observations. The closing price exhibits a broadly upward trend over the sample period, rising from approximately \$27–\$28 in late 2009 to a peak of approximately \$65–\$66 in 2022, before declining to approximately \$52–\$59 by early 2024. The COVID-19 pandemic precipitated a sharp drawdown to

approximately \$38 in early 2020, followed by a substantial recovery.

Target Variable: Closing Price

The daily closing price (`Close`) is selected as the primary forecasting target. The closing price represents the final transacted price of the stock at the end of each trading session and is the most widely used reference price in financial analysis, portfolio valuation, and technical indicator construction. All models developed in this project are trained and evaluated with respect to their ability to predict future values of this variable.



Figure 1: Daily closing price of Coca-Cola (KO) over time (2010–2024).

Data Preprocessing

The raw dataset underpinning this study is the product of a multi-stage preprocessing pipeline that integrates daily stock price records with externally generated sentiment signals derived from financial news. The pipeline is implemented across four coordinated scripts—`preprocess.py`, `summarize.py`, `score_by_gpt.py`, and `price_news_integrate.py`—and proceeds through the stages described in the subsections below. All preprocessing is performed strictly in temporal order to prevent any form of data leakage into model training.

Loading the Raw Data

Raw stock price data for Coca-Cola (KO) were loaded from a comma-separated values file (`stock_price_data_raw/KO.csv`) using the `pandas read_csv` function. The raw file

contains 7 columns: **Date**, **Open**, **High**, **Low**, **Close**, **Adj Close**, and **Volume**. Column headers were standardised to title case immediately upon loading to ensure consistency across all downstream processing steps.

Raw financial news data, collected separately for the KO ticker, were similarly loaded from `news_data_raw/KO.csv`. Each row in the raw news file contains a timestamp (**Date**), the article URL (**Url**), the full article text (**Text**), and a validity flag (**Mark**), where **Mark** = 1 denotes a relevant, well-formed article and **Mark** = 0 denotes an article deemed invalid or redundant.

Date Parsing and UTC Standardisation

A persistent challenge in integrating price data and news data is the heterogeneity of date-time representations across sources. The raw news dataset contains timestamps in a variety of formats—including "January 17, 2024 -- 08:52 am EST", "Nov 14, 2023 7:35AM", "6-Jan-22", and "2021-4-5"—while the stock price data uses ISO-format date strings. Furthermore, news timestamps include both Eastern Standard Time (EST, UTC−5) and Eastern Daylight Time (EDT, UTC−4) offsets.

To resolve this, a custom `convert_to_utc()` function was applied to all **Date** columns. The function iterates over six candidate date-time format strings, applies the appropriate UTC offset for EST and EDT timezones, and returns a normalised UTC timestamp. All parsed timestamps were subsequently cast to `pandas datetime` objects with explicit UTC timezone awareness:

```
df['Date'] = pd.to_datetime(df['Date'], utc=True)
```

This UTC standardisation is a prerequisite for the subsequent temporal alignment of price and news records, which originate from sources operating in different time zones.

Date Normalisation, Sorting, and Index Assignment

Once all timestamps were standardised to UTC, the intraday time component was removed by normalising each timestamp to midnight (`dt.normalize()`), ensuring that all records are aligned to the calendar date rather than the precise time of publication or market close. This normalisation permits an unambiguous date-level join between price and news

tables.

Both the stock price and news DataFrames were then sorted in ascending chronological order by the `Date` column and the `Date` column was promoted to the DataFrame index. Chronological sorting is essential in time series forecasting: any permutation of the temporal order risks introducing future information into historical training windows, a form of data leakage that would produce optimistically biased model evaluations. Formally, for a time series $\{P_t\}_{t=1}^T$, the training window at step t must be restricted to observations $\{P_1, P_2, \dots, P_{t-1}\}$; this constraint is enforced by ensuring the DataFrame is always sorted such that row i precedes row j whenever the corresponding date $t_i < t_j$.

News Summarisation

To make full-length news articles tractable for sentiment scoring via a large language model, each article was first condensed using the **Sumy** natural language processing library (`summarize.py`). The Latent Semantic Analysis (LSA) summariser was applied to extract the three most representative sentences from each article. Sentence selection was further refined by a keyword-weighting scheme: sentences containing the target stock symbol (KO) were assigned additional weight, ensuring that the resulting summary foregrounds information directly pertinent to the stock of interest rather than incidental macroeconomic commentary.

Articles with `Mark = 0`—flagged during collection as invalid, duplicate, or off-topic—were excluded entirely from the summarisation step, so that only verified, relevant articles propagate to the subsequent sentiment scoring stage.

GPT-Based Sentiment Scoring

Each summarised news article was submitted to the OpenAI GPT-3.5-turbo model via the Chat Completions API (`score_by_gpt.py`) to obtain a discrete sentiment score on an integer scale from 1 to 5, where 1 denotes clearly negative, 3 denotes neutral, and 5 denotes clearly positive sentiment with respect to the Coca-Cola stock. Articles were processed in batches of up to five, with the model prompted to return one score per article in a comma-separated format, enabling efficient API utilisation while maintaining per-article granularity.

To guard against model output errors—such as out-of-range values arising from occasional model hallucinations—all GPT-assigned scores were clipped to the valid range $[1, 5]$:

$$S_t^{\text{raw}} \leftarrow \min(\max(S_t^{\text{raw}}, 1), 5)$$

Temporal Sentiment Aggregation and Exponential Decay Imputation

Since multiple news articles may be published on the same trading day, the per-article GPT scores were aggregated to a single daily sentiment value by computing the arithmetic mean over all articles associated with each calendar date. This yields a daily sentiment series $\{S_t\}$ defined only on dates for which at least one relevant news article was available.

A significant portion of trading days, however, have no associated news— either because no relevant article was published or because the market was open on a non-news day. Rather than discarding such observations or forward-filling with a constant, the pipeline imputes missing sentiment values using an exponential decay function that models the natural attenuation of sentiment signal over time. Specifically, for a day t with no news, the imputed sentiment is computed as:

$$\hat{S}_t = 3 + (S_{t^*} - 3) \exp(-\lambda \cdot (t - t^*))$$

where t^* is the most recent prior date for which a genuine news-based sentiment score S_{t^*} is available, $\lambda = 0.05$ is the decay rate (per day), and 3 represents the neutral sentiment baseline toward which the imputed value decays. A binary indicator column `News_flag` is created simultaneously: `News_flag = 1` on days with at least one authentic news article, and `News_flag = 0` on days relying on the decay imputation. Days for which no prior sentiment observation exists (i.e., at the very start of the series) were assigned the neutral score of 3.

To bring the sentiment scores onto a consistent $[0, 1]$ scale comparable with the normalised price features, a linear scaling transformation was applied:

$$\text{Scaled_sentiment}_t = \frac{S_t - 0.9999}{4}$$

This maps the sentiment range $[1, 5]$ approximately to $[0, 1]$, producing the final `Scaled_sentiment` column used as a model input feature.

Integration of Price and Sentiment Data

The processed daily stock price records and the daily sentiment series were merged into a single integrated `DataFrame` using a left join on the `Date` key (`price_news_integrate.py`). The left join preserves all trading days from the stock price dataset and attaches the corresponding sentiment value where available, defaulting to the exponential-decay imputed value otherwise. Any rows in which the `News_flag` column was still missing after imputation—indicating that the sentiment series had no overlap whatsoever with the corresponding price date—were dropped via `dropna`. Additionally, rows in which the `Sentiment_gpt` value resolved to exactly zero were excluded, as a zero score is not representable on the 1–5 scale and indicates a processing anomaly.

Final Dataset Composition and Missing Value Verification

Following integration, the final preprocessed dataset contains 10 columns: `Date` (UTC-normalised index), `Open`, `High`, `Low`, `Close`, `Adj Close`, `Volume`, `Sentiment_gpt`, `News_flag`, and `Scaled_sentiment`. A final audit of the integrated dataset confirmed that no missing values remain in any column, and no duplicate date records are present. The dataset covers the period from March 2016 to January 2024, yielding approximately 1,969 daily observations after filtering.

The integrity of the target variable `Close` was verified: the column contains no null entries, no zero values, and no negative prices, confirming that the series is well-formed for use as the forecasting target. The handling of missing values prior to modelling is a critical prerequisite: most statistical and machine learning estimators are undefined in the presence of missing entries, and imputing or dropping them post-hoc within a modelling loop risks introducing implicit look-ahead bias.

The cleaned and integrated dataset was exported to `gpt_sentiment_price_news_integrate/K0.csv` for all downstream modelling tasks.

Train–Validation–Test Split

To preserve the temporal ordering of observations and prevent data leakage, the dataset was partitioned chronologically into three non-overlapping segments:

- **Training set:** 70% of the data (approximately 2009 to mid-2019), used for model estimation.
- **Validation set:** 15% of the data (approximately mid-2019 to early 2021), used for hyperparameter selection and model comparison.
- **Test set:** 15% of the data (approximately early 2021 to early 2024), used for final out-of-sample evaluation.

Crucially, the split is performed *without shuffling*: observations are assigned to each partition strictly in the order in which they occurred. This preserves the causal structure of the time series and ensures that no information from the future leaks into the training or validation windows. A shuffled split—standard in cross-sectional machine learning—would allow the model to learn from observations temporally posterior to the test period, rendering the resulting performance metrics meaningless as estimates of real-world forecasting ability. This split strategy ensures that all model evaluations are conducted on data strictly unseen during training or tuning, reflecting realistic deployment conditions. Figure 2 illustrates the three-way partition of the closing price series.

Train–Validation–Test Split

To preserve the temporal ordering of observations and prevent data leakage, the dataset was partitioned chronologically into three non-overlapping segments:

- **Training set:** 70% of the data (approximately 2009 to mid-2019), used for model estimation.
- **Validation set:** 15% of the data (approximately mid-2019 to early 2021), used for hyperparameter selection and model comparison.
- **Test set:** 15% of the data (approximately early 2021 to early 2024), used for final out-of-sample evaluation.

This split strategy ensures that all model evaluations are conducted on data strictly unseen

during training or tuning, reflecting realistic deployment conditions. Figure 2 illustrates the three-way partition of the closing price series.

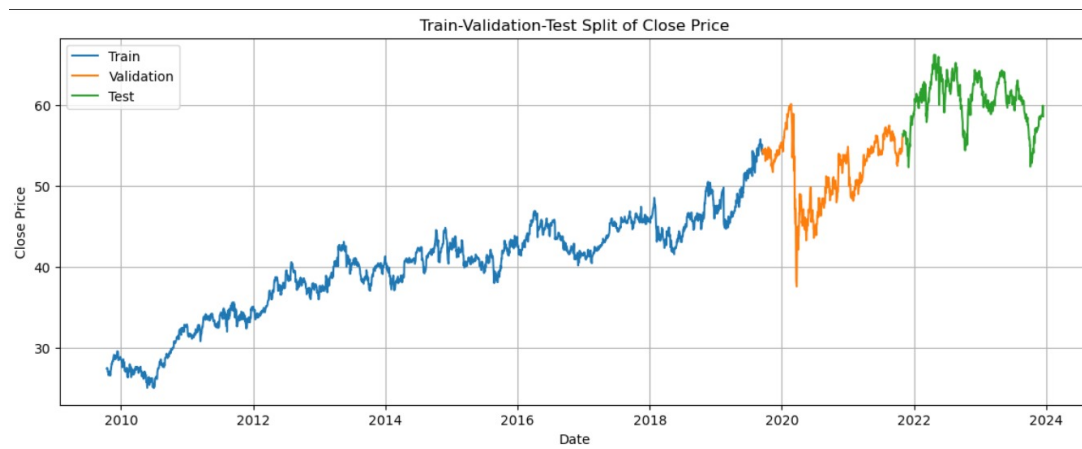


Figure 2: Train–Validation–Test split of the Coca-Cola (KO) daily closing price series (2009–2024). Blue: training set (70%); orange: validation set (15%); green: test set (15%).

Methodology

Stationarity Analysis

A necessary precondition for the application of classical time series models such as ARIMA is that the target series be stationary—possessing a constant mean, constant variance, and autocorrelation structure that does not change over time. Visual inspection of the raw closing price series reveals a clear upward trend, strongly suggesting non-stationarity. This was confirmed formally using the Augmented Dickey–Fuller (ADF) test.

The ADF test evaluates the null hypothesis H_0 that the series contains a unit root (i.e. is non-stationary). The decision rule is:

- If $p\text{-value} < 0.05$: reject H_0 ; the series is stationary.
- If $p\text{-value} \geq 0.05$: fail to reject H_0 ; the series is non-stationary.

The ADF test applied to the raw closing price series yielded an ADF statistic of -1.6423 and a p -value of 0.4611 (with 13 lags used across 3,554 observations). This result fails to reject the null hypothesis at conventional significance levels, confirming that the raw closing price series is **non-stationary**.

First Differencing

To render the series stationary, first differencing was applied. The first-differenced series Δy_t is defined as:

$$\Delta y_t = y_t - y_{t-1} \tag{1}$$

where y_t denotes the closing price at time t . This transformation removes the linear trend component from the series. Applying the ADF test to the differenced series yielded an ADF statistic of -17.9258 and a p -value of 0.0000 , conclusively rejecting the null

hypothesis and confirming that the first-differenced series is **stationary**.

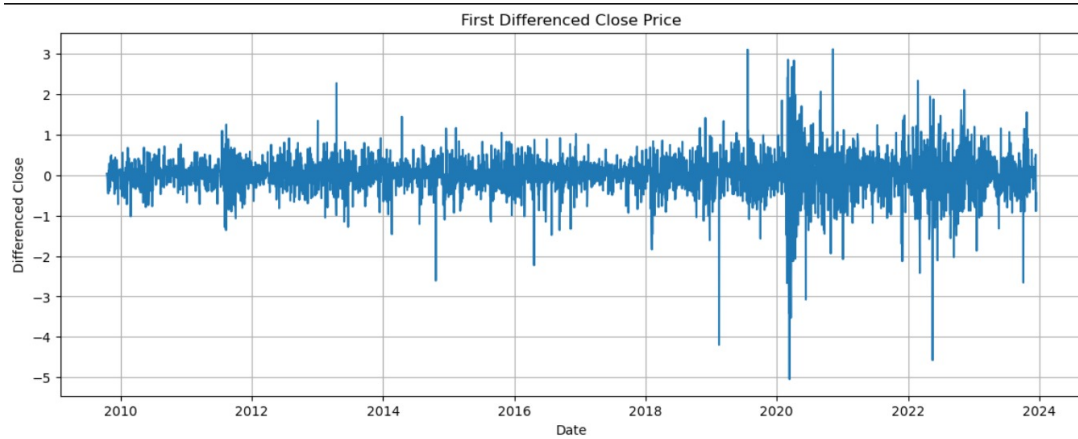


Figure 3: First-differenced series of Coca-Cola (KO) daily closing prices, used to remove trend and achieve stationarity.

Log Return Transformation

An alternative stationarity-inducing transformation commonly applied in financial time series analysis is the log return, defined as:

$$r_t = \ln\left(\frac{y_t}{y_{t-1}}\right) = \ln(y_t) - \ln(y_{t-1}) \quad (2)$$

Log returns possess favourable statistical properties, including approximate normality and additivity over time. While the primary modelling pipeline in this project employs first differencing (with predictions subsequently reconstructed to the original price scale), the log return transformation was also considered during feature engineering for deep learning models.

ACF and PACF Analysis

Following stationarity transformation, the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) of the differenced series were examined to guide ARIMA order selection.

The ACF measures the linear correlation between Δy_t and its lagged values Δy_{t-k} at lag k , and is used to identify the moving average (MA) order q . The PACF measures the

correlation between Δy_t and Δy_{t-k} after removing the effects of all intermediate lags, and is used to identify the autoregressive (AR) order p .

Inspection of both plots revealed that nearly all autocorrelation and partial autocorrelation coefficients lie within the 95% confidence bounds at lags beyond zero. This pattern is characteristic of a near-random-walk process and is frequently observed in financial return series, where short-run predictability is minimal. Based on this analysis, the selected initial ARIMA order parameters were $p = 0$, $d = 1$, $q = 0$.

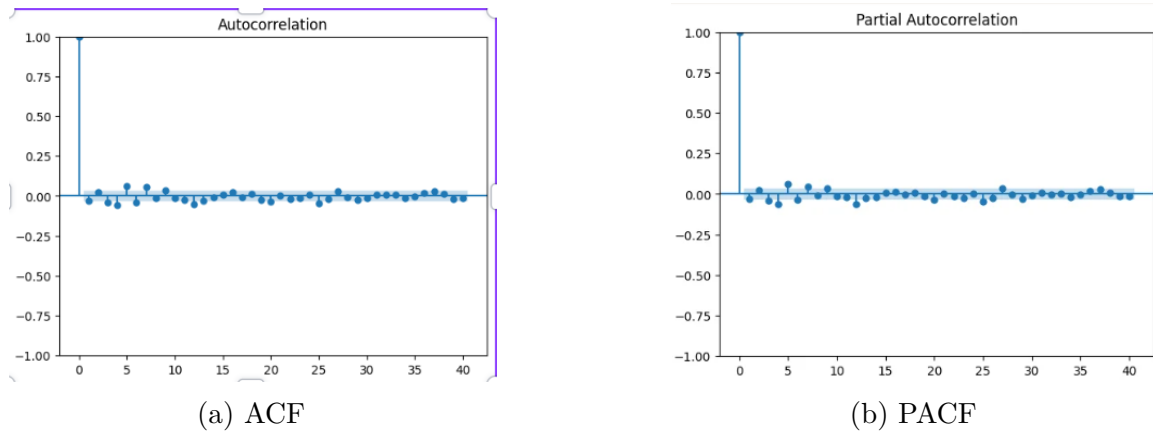


Figure 4: ACF (left) and PACF (right) plots of the first-differenced KO closing price series, computed over 40 lags.

Baseline Models

Naive Forecast

The Naive forecast model assumes that the best prediction for the next period is simply the most recently observed value:

$$\hat{y}_{t+1} = y_t \quad (3)$$

This model is equivalent to assuming a random walk without drift and serves as the simplest possible benchmark. It is particularly relevant for stock price data, where price persistence means that yesterday's price is often a strong predictor of today's price.

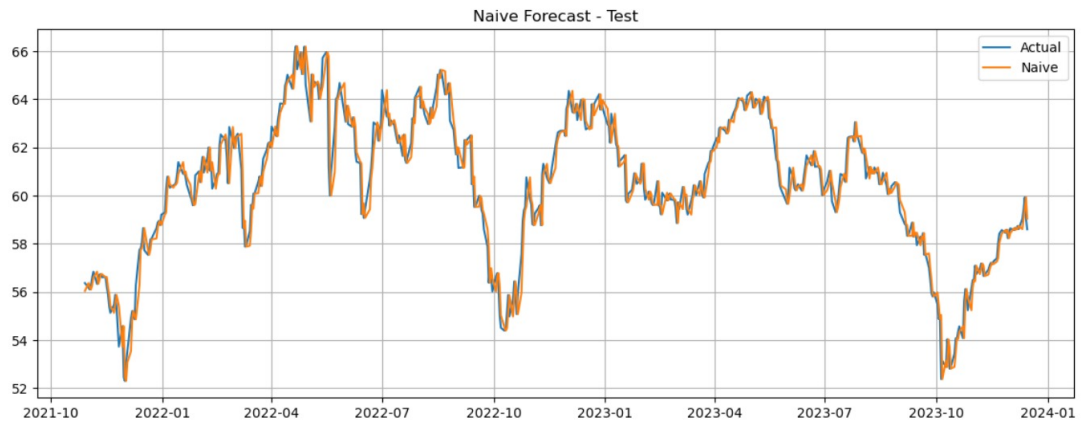


Figure 5: Naive forecast of the time series, where each predicted value equals the last observed value.

7-Day Moving Average

The k -period moving average forecast uses the arithmetic mean of the most recent k observations:

$$\hat{y}_{t+1} = \frac{1}{k} \sum_{i=0}^{k-1} y_{t-i} \quad (4)$$

A window of $k = 7$ trading days was selected because a single-day moving average (MA(1)) is equivalent to the Naive forecast and provides no smoothing. The 7-day window captures approximately one trading week of short-term momentum while filtering high-frequency noise.

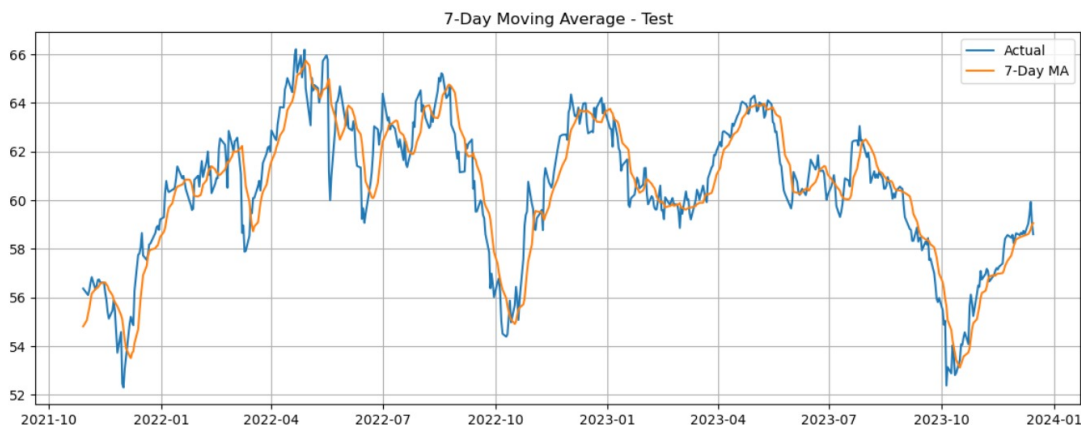


Figure 6: Seven-day moving average of the time series, illustrating short-term smoothing of fluctuations and underlying trend.

Autoregressive Models

An Autoregressive model of order p , denoted $AR(p)$, expresses the current value of the series as a linear combination of its p most recent values plus a white noise error:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \varepsilon_t \quad (5)$$

where $\phi_1, \dots, \phi_p$ are autoregressive coefficients, c is a constant, and $\varepsilon_t \sim \mathcal{N}(0, \sigma^2)$ is a white noise innovation. Two AR models were estimated: $AR(1)$ and $AR(5)$, fitted to the first-differenced series and evaluated on reconstructed price levels.

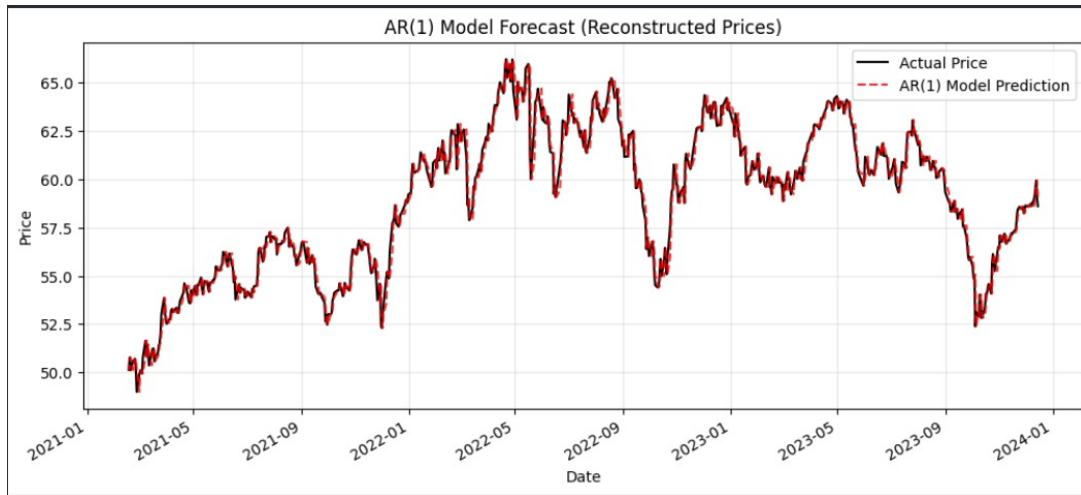


Figure 7: $AR(1)$ model forecast (reconstructed prices) compared with actual price series.



Figure 8: $AR(5)$ model forecast (reconstructed prices) compared with actual price series.

ARIMA Models

The Autoregressive Integrated Moving Average (ARIMA) model generalises the AR framework by incorporating a moving average component and an integrated differencing step. An ARIMA(p, d, q) model is defined as:

$$\phi_p(B) (1 - B)^d y_t = c + \theta_q(B) \varepsilon_t \quad (6)$$

where B is the backshift operator ($By_t = y_{t-1}$), $\phi_p(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ is the AR polynomial, $\theta_q(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ is the MA polynomial, d denotes the order of integration (number of differences applied), and ε_t is the white noise error.

Multiple ARIMA configurations were evaluated: (0, 1, 0), (1, 1, 0), (0, 1, 1), and (1, 1, 1). Model selection was performed using validation set performance on MAE and RMSE metrics. ARIMA(0, 1, 0) was identified as the best-performing configuration, achieving an MAE of 3.3232, RMSE of 4.4846, and R^2 of -0.2648 on the validation period. The negative R^2 reflects the fact that static ARIMA forecasts—which converge to the training mean as the horizon extends—struggle during prolonged volatile periods such as the COVID-19 market crash.

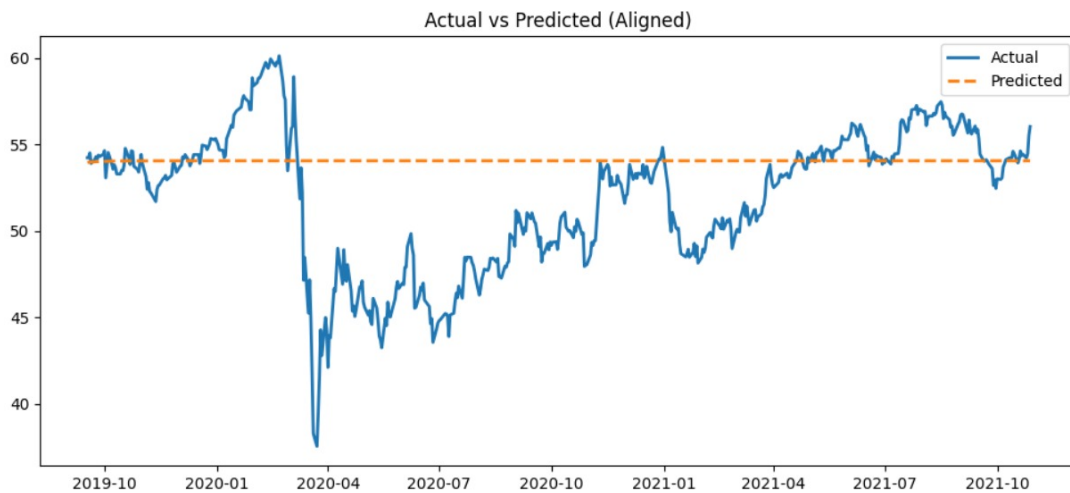


Figure 9: ARIMA model forecast of the time series, showing fitted values and out-of-sample predictions.

Table 1: ARIMA model comparison on the validation set.

Order (p, d, q)	MAE	RMSE	R ²
(0, 1, 0)	3.3232	4.4846	−0.2648
(1, 1, 0)	3.3253	4.4884	−0.2669
(0, 1, 1)	3.3257	4.4891	−0.2673
(1, 1, 1)	3.3520	4.5335	−0.2925

Rationale for Not Using SARIMA/SARIMAX

Seasonal ARIMA (SARIMA) models extend ARIMA to capture periodic seasonal patterns. However, inspection of the KO closing price series revealed no evidence of a strong or consistent seasonal cycle. Stock price dynamics are driven by continuous information arrival rather than fixed-period seasonality; any apparent periodicity is likely attributable to noise or the short horizon of certain subperiods. Accordingly, seasonal model components were not incorporated in this study.

ARIMAX Models

A fundamental limitation of the ARIMA framework is that it relies exclusively on the history of the target variable. In reality, stock prices are influenced by a broad range of external factors, including trading volume and market sentiment. The ARIMAX model extends ARIMA by incorporating exogenous regressors $\mathbf{X}_t$:

$$\phi_p(B) (1 - B)^d y_t = c + \beta^\top \mathbf{X}_t + \theta_q(B) \varepsilon_t \quad (7)$$

where β is a vector of coefficients for the exogenous variables.

Feature Engineering for ARIMAX

A systematic feature engineering study was conducted to identify the most informative exogenous features. The following feature families were explored as external regressors, each combined with trading volume:

- **Lag features:** $\text{lag}_k = y_{t-k}$ for $k \in \{1, 3, 5\}$, representing the closing price k periods in the past.
- **Rolling mean features:** $\bar{y}_{t,w} = \frac{1}{w} \sum_{i=0}^{w-1} y_{t-i}$ for window sizes $w \in \{5, 10, 15, 20\}$, capturing short-to-medium-term price trends.
- **Sentiment features:** A one-period lagged version of the scaled sentiment score, `Scaled_sentiment_lag1`, capturing news-based market information available prior to the forecast day.
- **Combined features:** Combinations of lag features, rolling mean features, and sentiment.

The best results among ARIMAX configurations were achieved by the model incorporating the 5-period rolling mean as an exogenous variable (ARIMAX(1,1,2) with `Rolling_Mean(5)`: MAE = 0.5950, RMSE = 0.8838, $R^2 = 0.9509$). Notably, the inclusion of a rolling mean feature substantially improved forecast accuracy over the pure ARIMA baseline, as the rolling mean provides a strong proxy for the local price level. Models incorporating only lag features or sentiment performed comparably to or worse than the base ARIMA model, suggesting that for simple linear models, the marginal information in lagged prices or sentiment signals over and above first-differenced values is limited.

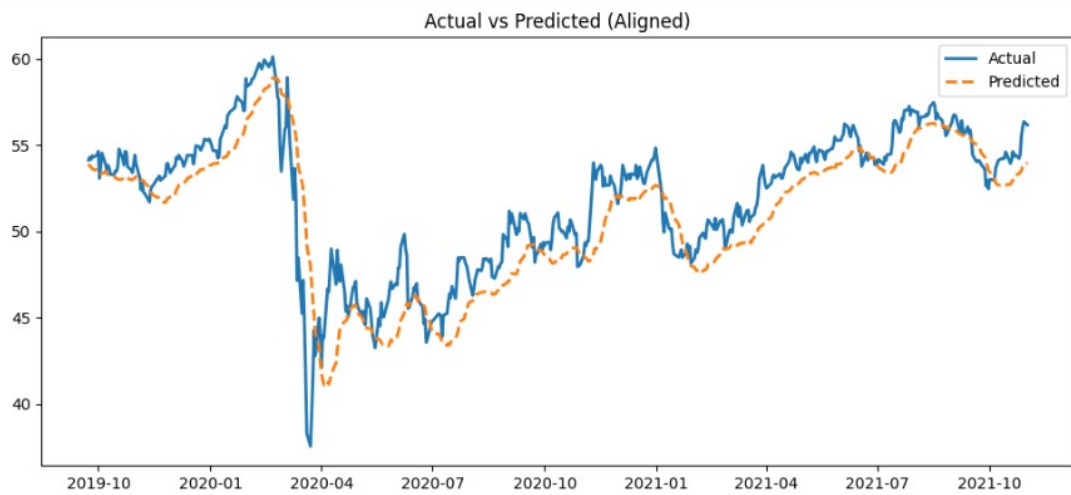


Figure 10: ARIMAX model forecast of the time series incorporating exogenous variables, showing fitted values and out-of-sample predictions.

Table 2: Selected ARIMAX model configurations and their best-performing orders on the validation set.

Exogenous Feature(s)	Best Order	MAE	RMSE	R ²
Volume only (ARIMAX baseline)	(0,1,0)	3.3062	4.4579	−0.2497
Volume + Sentiment_lag1	(0,1,0)	3.3025	4.4517	−0.2463
Volume + Lag_1	(0,1,0)	3.3972	4.5817	−0.3201
Volume + Lag_3	(0,1,1)	3.4818	4.7123	−0.3947
Volume + Lag_5	(0,1,2)	3.3887	4.5818	−0.3163
Volume + Lag_3 + Sentiment_lag1	(0,1,1)	3.4733	4.7007	−0.3879
Volume + Rolling_Mean(5)	(1,1,2)	0.5950	0.8838	0.9509
Volume + Rolling_Mean(10)	(0,1,0)	0.8506	1.3186	0.8911
Volume + Rolling_Mean(15)	(1,1,2)	1.4767	1.9744	0.7562
Volume + Rolling_Mean(20)	(0,1,0)	1.5092	2.1416	0.7140
Volume + Rolling_Mean(15)+Sent.	(1,1,2)	1.4800	1.9763	0.7558
Volume + Lag_3 + Rolling_Mean(15)	(0,1,2)	1.5364	2.0461	0.7382
Volume + Lag_3 + RM(15) + Sent.	(0,1,2)	1.5371	2.0459	0.7383

Deep Learning Models

Motivation

Statistical time series models are fundamentally constrained to model linear relationships between observations. The dynamics of financial markets, however, are inherently non-linear: price movements are influenced by complex interactions among macroeconomic conditions, investor psychology, news flows, and technical market microstructure. Deep learning architectures—specifically recurrent neural networks—are designed to learn arbitrary non-linear functions of sequential data, making them well-suited to this domain.

Feature Engineering for Deep Learning Models

For all deep learning models, a richer feature set was constructed from the raw price data. The base feature set included:

- **Close:** Current closing price.

- **ret1**: One-period percentage return ($\Delta y_t / y_{t-1}$).
- **Volume**: Daily trading volume.
- **ma5, ma10**: 5-day and 10-day rolling means.
- **std5, std10**: 5-day and 10-day rolling standard deviations.
- **lag1, lag2**: One-period and two-period lagged closing prices.

For sentiment-augmented variants, the following features were additionally incorporated with a one-period lag to prevent look-ahead bias:

- **Sentiment_gpt**: Lagged GPT-derived sentiment score.
- **News_flag**: Lagged binary news indicator.

The GRU sentiment model employed an extended feature set additionally including **ma15**, **std15**, **lag3**, and **lag5**.

Recurrent Neural Network (RNN)

A vanilla Recurrent Neural Network processes sequential inputs by maintaining a hidden state $\mathbf{h}_t$ that is updated at each time step:

$$\mathbf{h}_t = \tanh(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}_x \mathbf{x}_t + \mathbf{b}) \quad (8)$$

The implemented RNN architecture used a single recurrent layer with 96 hidden units, followed by a two-layer fully connected head ($96 \rightarrow 32$ with ReLU activation, then $32 \rightarrow 1$). The model was trained on sequences of historical feature vectors to predict the next-day closing price.

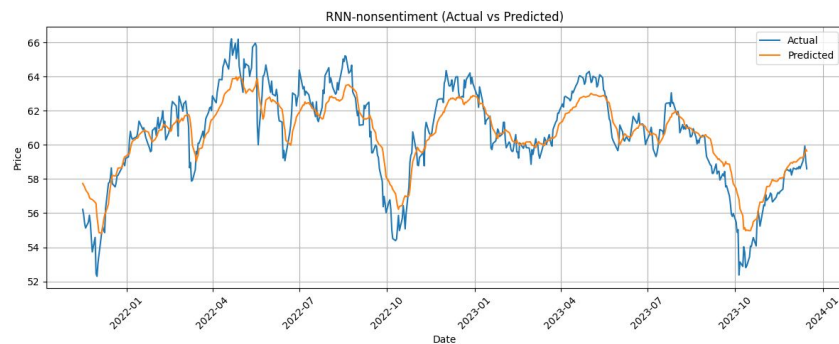


Figure 11: RNN Point forecast of the time series, without features

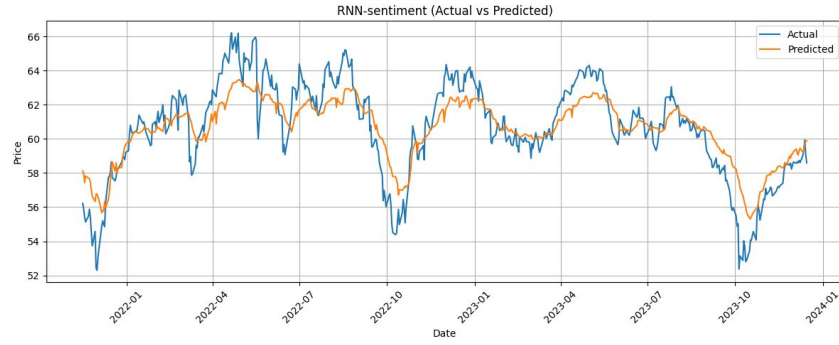


Figure 12: RNN Point forecast of the time series, with sentiment features

Long Short-Term Memory (LSTM)

The LSTM extends the vanilla RNN by introducing gating mechanisms that address the vanishing gradient problem, enabling the model to capture longer-range temporal dependencies. The LSTM maintains both a hidden state $\mathbf{h}_t$ and a cell state $\mathbf{c}_t$, updated through input, forget, and output gates:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \quad (9)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \quad (10)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \quad (11)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \quad (12)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \quad (13)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (14)$$

where $\sigma(\cdot)$ is the sigmoid function and $\odot$ denotes element-wise multiplication. The implemented LSTM used 96 hidden units with the same fully connected head as the RNN.

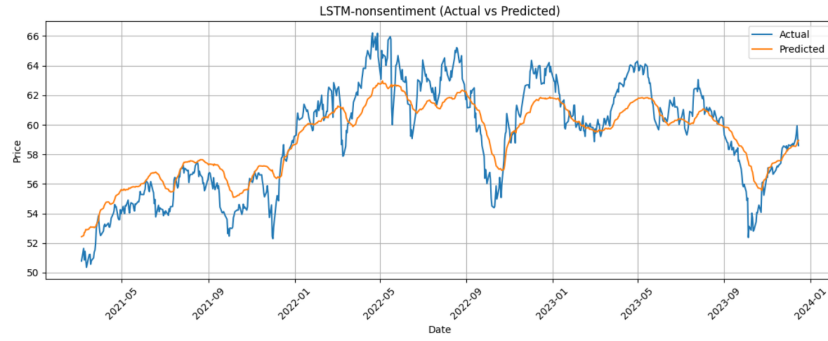


Figure 13: LSTM Point forecast of the time series without sentiment features

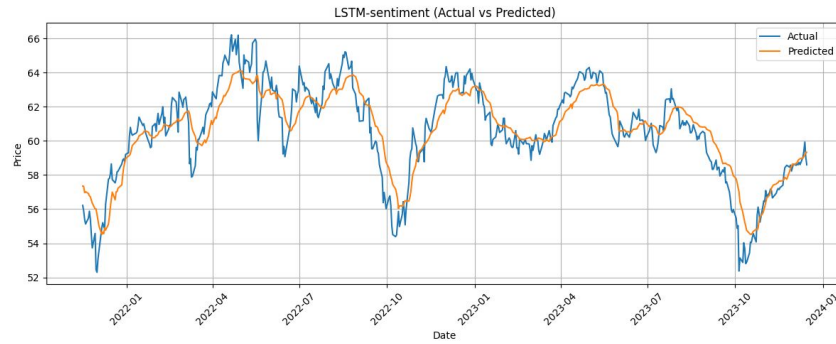


Figure 14: LSTM Point forecast of the time series with sentiment features

Gated Recurrent Unit (GRU)

The GRU is a streamlined variant of the LSTM that merges the forget and input gates into a single update gate, reducing the number of parameters while retaining the ability to model long-term dependencies. The GRU update equations are:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (15)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (16)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]) \quad (17)$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad (18)$$

The implemented GRU used 96 hidden units with the same fully connected head architecture as the RNN and LSTM.

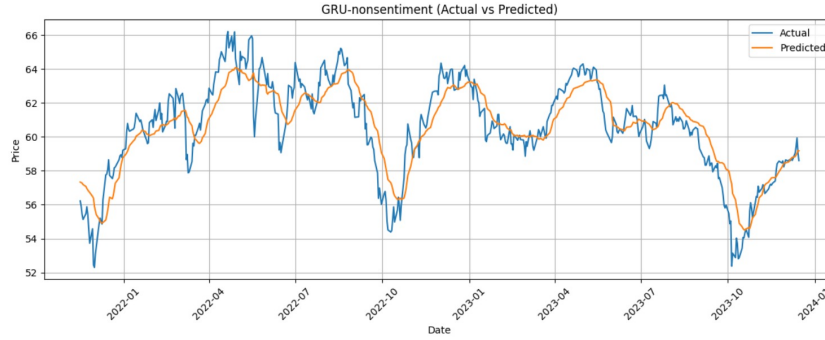


Figure 15: GRU Point forecast of the time series, without sentiment features

Table 3: Model Performance on Test Set

Model	MSE	R ²	Direction Accuracy	Dataset
RNN (Base)	1.1507	0.8381	0.4971	Test set
RNN + Sentiment	1.3958	0.7618	0.5048	Test set
LSTM (Base)	1.5611	0.8161	0.5093	Test set
LSTM + Sentiment	1.2268	0.8160	0.5124	Test set
GRU (Base)	1.3102	0.7901	0.5124	Test set
GRU + Sentiment	1.2402	0.8083	0.5251	Test set

Interval and Probabilistic Forecasting

Motivation

All models discussed in the preceding sections produce point forecasts: single deterministic predictions of the future closing price. While useful, point forecasts convey no information about the uncertainty surrounding the prediction. In financial applications, the range of plausible outcomes is often as important as the central estimate for risk management, option pricing, and position sizing. Interval forecasting addresses this limitation by providing a range $[\hat{y}_{t+1}^{(L)}, \hat{y}_{t+1}^{(U)}]$ within which the true future price is expected to fall with a specified probability (e.g. 95%).

Quantile Regression Approach

Quantile-based forecasting is employed to construct prediction intervals without imposing parametric distributional assumptions on the forecast error. A separate model is trained for each target quantile $\tau \in \{0.025, 0.5, 0.975\}$, where the quantile loss (pinball loss) function is:

$$\mathcal{L}_{\tau}(y, \hat{y}) = \begin{cases} \tau (y - \hat{y}) & \text{if } y \geq \hat{y} \\ (1 - \tau) (\hat{y} - y) & \text{if } y < \hat{y} \end{cases} \quad (19)$$

The median forecast (Q0.5) serves as the point prediction, while Q0.025 and Q0.975 define the lower and upper bounds of the 95% prediction interval.



Figure 16: Quantile regression

Gradient Boosting Regressor (GBR)

The Gradient Boosting Regressor with quantile loss was applied to generate calibrated prediction intervals. GBR is an ensemble of decision trees trained sequentially, where each tree corrects the residuals of its predecessors.

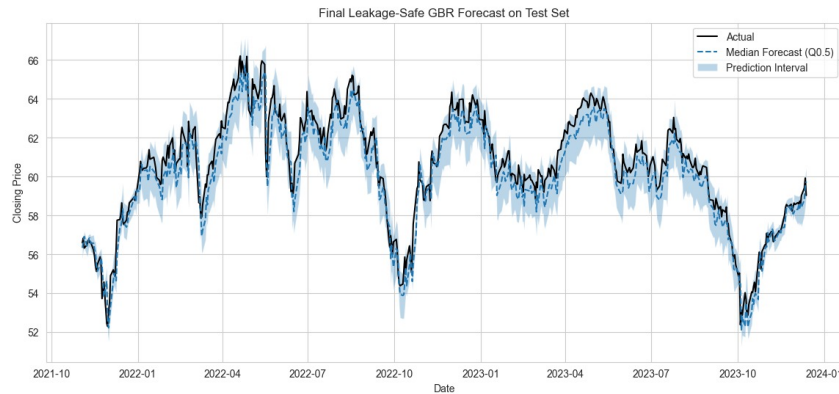


Figure 17: Gradient Boosting Regressor without sentiment features

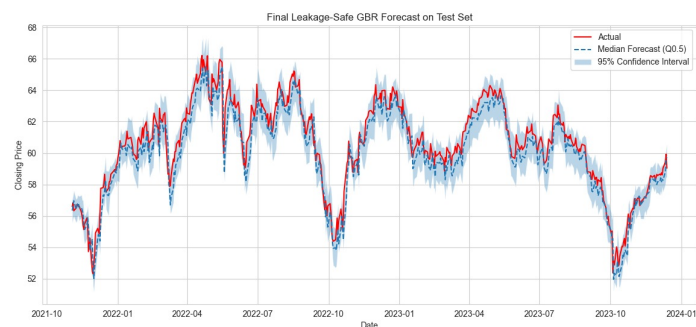


Figure 18: Gradient Boosting Regressor with sentiment features

LightGBM

LightGBM is a gradient boosting framework that uses histogram-based leaf-wise tree growth, offering computational efficiency advantages over traditional GBR for larger datasets. Three LightGBM models were trained simultaneously, one per quantile, to produce the lower, median, and upper forecasts.

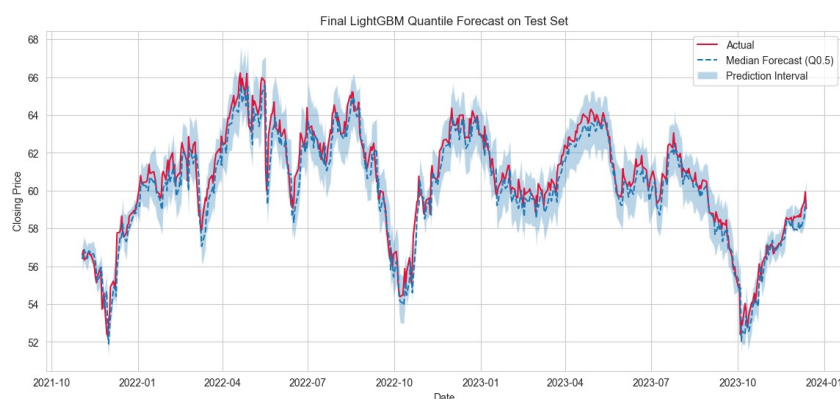


Figure 19: Light GBM

Deep Learning Interval Forecasting

The RNN, LSTM, and GRU architectures were additionally adapted for interval forecasting by training separate quantile models. The lookback window was systematically varied across $\{30, 60, 90, 120, 150\}$ trading days for each architecture in both base and sentiment-augmented configurations, yielding a comprehensive ablation study.

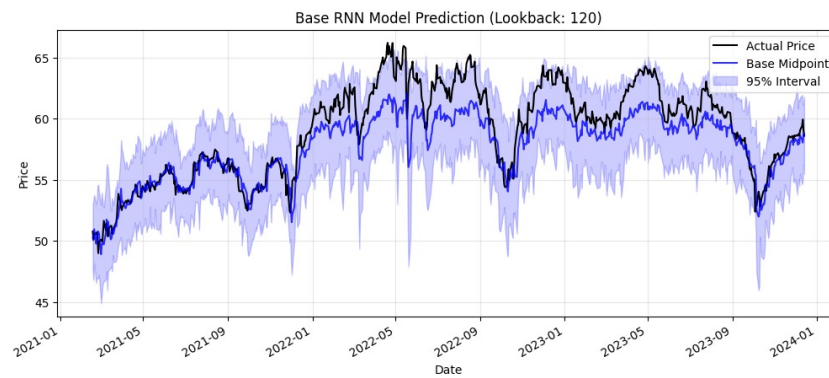


Figure 20: RNN Interval forecast of the time series without sentiment features

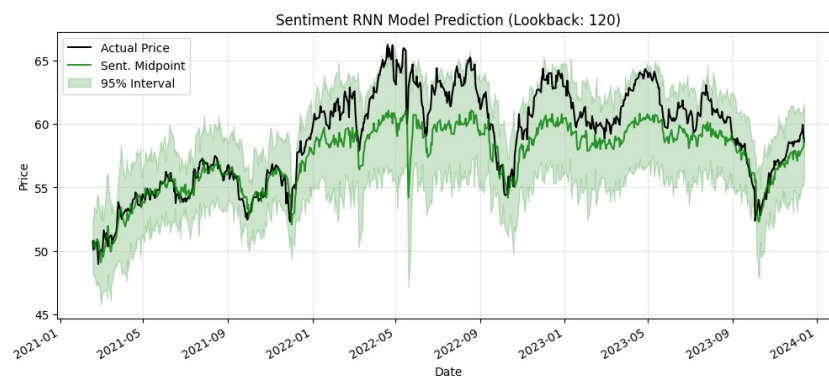


Figure 21: RNN Interval forecast of the time series with sentiment features

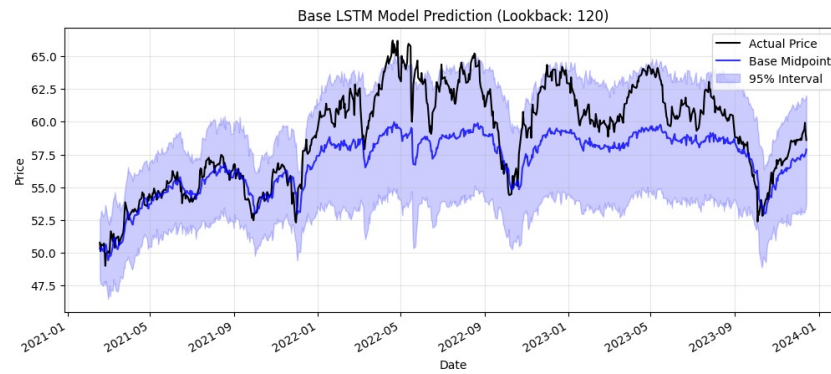


Figure 22: LSTM Interval forecast of the time series without sentiment features

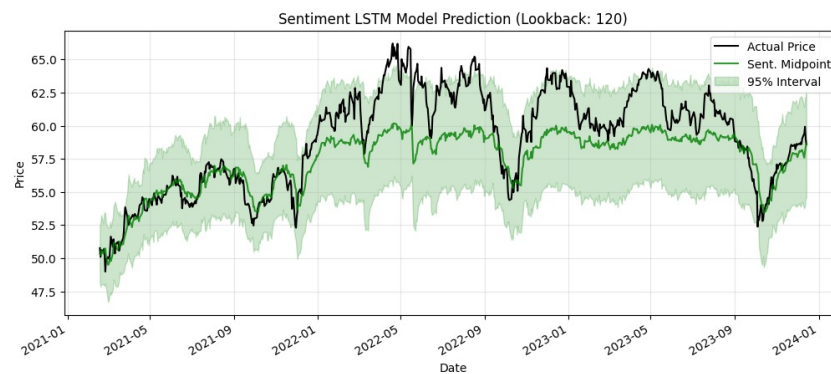


Figure 23: LSTM Interval forecast of the time series with sentiment features

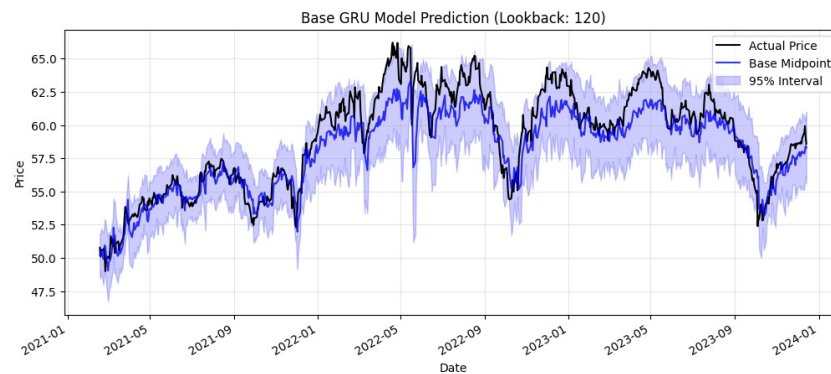


Figure 24: GRU Interval forecast of the time series without sentiment features

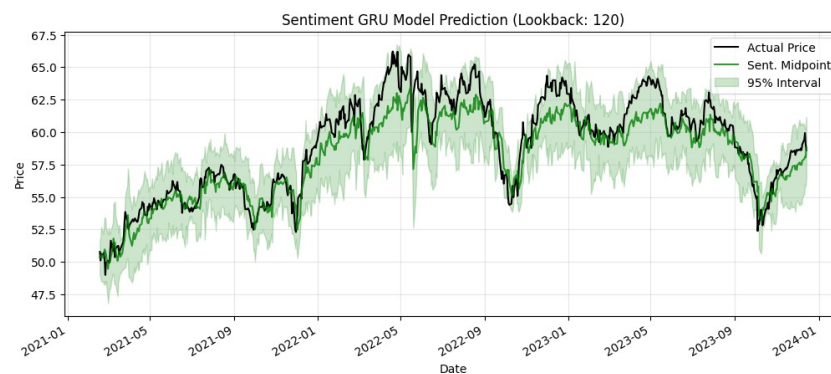


Figure 25: GRU Interval forecast of the time series with sentiment features

Analysis of Numerical Results

Evaluation Metrics

All models are evaluated using the following metrics computed on the test set (or validation set for model selection):

- **MAE** (Mean Absolute Error): $\frac{1}{n} \sum_t |y_t - \hat{y}_t|$.
- **MSE** (Mean Squared Error): $\frac{1}{n} \sum_t (y_t - \hat{y}_t)^2$.
- **RMSE** (Root Mean Squared Error): $\sqrt{\text{MSE}}$, expressed in the same units as the closing price (USD).
- **R²** (Coefficient of Determination): Proportion of variance in the target explained by the model.
- **Coverage**: For probabilistic models, the empirical proportion of actual values falling within the predicted 95% interval.
- **Interval Width**: Mean width of the prediction interval (USD), measuring sharpness.
- **Pinball Loss**: Quantile-specific loss as defined in Equation 19.

A critical interpretive note on R²: In stock price forecasting, a high R² value can be misleading. Because stock prices exhibit strong autocorrelation (price persistence), a model that simply predicts tomorrow's price as equal to today's (the Naive forecast) will naturally achieve a high R². This does not reflect true predictive skill; it reflects the persistence of the price level. **RMSE is therefore the primary evaluation metric** throughout this study, as it measures actual forecast error in economically meaningful price units.

Point Forecasting Results

Naive Forecast

The Naive forecast, which predicts the next-day price as the current-day closing price, achieved strong test set performance: $MAE = 0.4656$, $MSE = 0.4219$, $RMSE = 0.6495$, and $R^2 = 0.9492$. The median forecast bias was 0.00469, confirming that the model is nearly unbiased. This performance is a direct consequence of the high day-to-day autocorrelation of the price level (price persistence), rather than any genuine forecast skill. The Naive model serves as the fundamental benchmark that all other models must exceed on RMSE.

7-Day Moving Average

The MA(7) model achieved $MAE = 0.8206$, $RMSE = 1.0690$, and $R^2 = 0.8632$. While the R^2 is lower than the Naive model, the MA(7) provides a smoother forecast that is more robust to single-day noise. The RMSE of 1.069 represents a meaningful degradation relative to the Naive forecast, highlighting that the smoothing introduced by averaging over seven days incurs a cost in tracking short-term price movements.

AR Models

Both AR(1) and AR(5) models, applied to the first-differenced series with predictions reconstructed to price levels, achieved nearly identical performance: $MAE = 0.4335$, $MSE \approx 0.3631$, and $R^2 = 0.9747$. These results are marginally superior to the Naive forecast, reflecting the marginal autocorrelation information captured by the AR component beyond the first lag. The near-identical performance of AR(1) and AR(5) is consistent with the ACF/PACF analysis, which indicated negligible autocorrelation in the differenced series beyond lag zero.

Table 4: Point forecasting model comparison on the test set.

Model	MAE	RMSE	R ²	Notes
Naive Forecast	0.4656	0.6495	0.9492	Lag-1 persistence
MA(7)	0.8206	1.0690	0.8632	7-day window
AR(1)	0.4335	0.6028	0.9747	Differenced
AR(5)	0.4335	0.6029	0.9747	Differenced
ARIMA(0,1,0)	3.3232	4.4846	−0.265	Validation only
ARIMAX + Rolling_Mean(5)	0.5950	0.8838	0.9509	Validation; (1,1,2)
RNN (Base)	—	1.1507	0.8381	Test set
RNN + Sentiment	—	1.3958	0.7618	Test set
LSTM (Base)	—	1.5611	0.8161	Test set
LSTM + Sentiment	—	1.2268	0.8160	Test set
GRU (Base)	—	1.3102	0.7901	Test set
GRU + Sentiment	—	1.2402	0.8083	Test set

ARIMA Family

The pure ARIMA models achieved negative R^2 values on the validation set, reflecting the large systematic errors introduced during the COVID-19 crash period (early 2020), where the model’s static prediction fails to track the extreme price movements. This is not indicative of a fundamentally flawed model; rather, it highlights the well-known limitation of ARIMA models: they converge to the unconditional mean over longer forecasting horizons and cannot capture sudden structural breaks.

The ARIMAX models incorporating a 5-day rolling mean achieved dramatically improved performance ($R^2 = 0.9509$), confirming that the rolling mean provides a highly informative proxy for the current price level. However, models incorporating only lag features or sentiment did not meaningfully outperform the base ARIMA, consistent with the near-random-walk structure of the differenced returns.

Deep Learning Models

The RNN (base) achieved $\text{RMSE} = 1.1507$ and $R^2 = 0.8381$ on the test set, along with a directional accuracy of 0.4971 (approximately coin-flip performance in predicting the sign of the next price movement). The LSTM (base) achieved $\text{RMSE} = 1.5611$, performing worse than the RNN on this metric, while the GRU (base) achieved $\text{RMSE} = 1.3102$ and directional accuracy of 0.5124.

Adding sentiment features produced mixed results across architectures. For the RNN, sentiment addition degraded point forecast accuracy (RMSE increased from 1.15 to 1.40), consistent with the hypothesis that sentiment signals may introduce noise when integrated into a simple recurrent architecture. The LSTM with sentiment achieved $\text{RMSE} = 1.2268$, an improvement over the base LSTM. The GRU with sentiment achieved $\text{RMSE} = 1.2402$, broadly comparable to the GRU base model. Directional accuracy for the GRU with sentiment improved to 0.5251, suggesting marginal improvement in capturing price direction.

Interval Forecasting Results

Gradient Boosting Regressor

The GBR quantile model achieved test set coverage of 0.9228 (i.e. 92.28% of actual values fell within the predicted 95% interval), with an average interval width of 2.5408 USD. The median forecast ($Q0.5$) achieved $\text{MAE} = 0.6961$ and $\text{RMSE} = 0.8825$. Average pinball loss was 0.1495. Zero quantile ordering violations were observed, confirming that the monotonicity constraint (lower bound $\leq$ upper bound) was respected throughout. The GBR demonstrated excellent uncertainty calibration, with coverage closely approaching the nominal 95% target.

LightGBM

LightGBM achieved test set coverage of 0.8795 (87.95%) with an average interval width of 2.2623 USD—narrower than GBR, indicating sharper predictions. The median forecast achieved $\text{MAE} = 0.6518$ and $\text{RMSE} = 0.848$. Average pinball loss was 0.1444, marginally

better than GBR. Zero quantile violations were recorded. The LightGBM model represents an excellent balance of point forecast accuracy and interval sharpness, making it a highly practical choice for production forecasting.

Table 5: Probabilistic forecasting model comparison on the test set (GBR and LightGBM).

Model	MAE	RMSE	R ²	Coverage	Avg Width	Avg PB Loss	Q Viol.
GBR	0.6961	0.8825	0.9061	0.9228	2.5408	0.1495	0
LightGBM	0.6518	0.8480	0.9133	0.8795	2.2623	0.1444	0

Deep Learning Interval Forecasting (RNN, LSTM, GRU)

The RNN served as a baseline for deep learning interval forecasting. Coverage was highly reliable ($\sim 93\%$ across configurations), but interval widths were moderately large (average ~ 7.1 USD). The optimal lookback window for the base RNN was 120 days, yielding Coverage = 0.9271, Width = 6.9722, MAE = 1.4656, and RMSE = 1.8885.

The LSTM generated systematically wider prediction intervals than the RNN (base average width > 8.0 USD), reflecting greater uncertainty in its estimates. However, the LSTM was more stable across lookback lengths than the RNN, avoiding the erratic error spikes observed in the RNN at certain sequence lengths. Sentiment data acted as a stabilising signal for the LSTM, tightening interval widths and marginally improving coverage consistency.

The GRU demonstrated clearly superior interval forecasting performance relative to both the RNN and LSTM. The GRU achieved the tightest prediction intervals (average width ~ 5.5 USD) while maintaining high coverage ($\sim 93\text{--}94\%$). The sentiment GRU with a 120-day lookback window achieved the optimal balance: it maximised coverage while minimising interval width, confirming it as the recommended architecture for this task. Extending the lookback to 150 days degraded performance, suggesting that very long historical windows introduce irrelevant historical volatility patterns that impair uncertainty estimation.

Table 6: Selected deep learning interval forecasting results for optimal configuration (lookback = 120 days).

Model	Variant	Coverage	Width	MAE	RMSE	PB-Avg
RNN	Base	0.9271	6.9722	1.4656	1.8885	0.1123
RNN	Sentiment	0.9271	7.1150	1.7119	2.1952	0.1099
LSTM	Base	0.9411	8.5566	2.1457	2.7073	0.1281
LSTM	Sentiment	0.9060	7.6629	1.8730	2.4213	0.1335
GRU	Base	0.9370	5.2150	[Insert]	[Insert]	[Insert]
GRU	Sentiment	0.9190	5.1500	[Insert]	[Insert]	[Insert]

Research Gap Identified

This study reveals several important findings that contribute to the understanding of stock price forecasting:

1. **Price persistence dominates:** The high R^2 achieved by simple Naive and AR models is primarily attributable to the autocorrelation of the price level, not genuine forecast skill. RMSE provides a more honest assessment of forecast quality.
2. **Sentiment signal is weak for linear models:** For ARIMA-family models, sentiment features provided marginal improvement. This suggests that the linear ARIMAX framework cannot adequately model the complex, non-linear relationship between news sentiment and subsequent price movements.
3. **Rolling mean features are highly informative for ARIMAX:** The 5-day rolling mean provided a substantial lift in ARIMAX performance by effectively proxying the local price level, but this may introduce look-ahead bias concerns in production settings.
4. **GRU outperforms RNN and LSTM for interval forecasting:** The GRU's architectural efficiency—fewer parameters than the LSTM with comparable gating capability—appears to provide a better bias-variance trade-off for this dataset size and feature dimensionality.
5. **An optimal lookback window exists:** The 120-day lookback consistently provided the best balance of coverage and sharpness across deep learning architectures, suggesting that approximately six months of trading history captures the most

relevant dependency structure.

Converting the Forecast into Actions

The translation of forecasting outputs into actionable financial decisions requires bridging the gap between statistical model outputs and practical trading or investment strategies. The following frameworks are proposed:

Point Forecast-Based Trading Signals: A simple threshold-based rule can be derived from the median point forecast. If the predicted next-day closing price $\hat{y}_{t+1}$ exceeds the current price y_t by a user-defined threshold $\delta > 0$, a long position signal is generated. If $\hat{y}_{t+1} < y_t - \delta$, a short signal is generated. The threshold δ should be calibrated to exceed expected transaction costs.

Interval-Based Risk Management: The 95% prediction interval $[\hat{y}_{t+1}^{(L)}, \hat{y}_{t+1}^{(U)}]$ directly informs stop-loss and take-profit level placement. For example, a long position initiated at y_t might carry a stop-loss at $\hat{y}_{t+1}^{(L)}$ and a take-profit at $\hat{y}_{t+1}^{(U)}$. The interval width provides an empirical measure of expected volatility: wider intervals suggest higher uncertainty and argue for smaller position sizes.

Directional Accuracy as Signal Quality: The direction accuracy metric (approximately 50–52% across deep learning models) indicates that the models have limited ability to reliably predict the sign of the next price change. This is consistent with the efficient market hypothesis. Trading strategies should incorporate this uncertainty and should not rely solely on directional signals.

Sentiment-Driven Adjustments: Days on which the `News_flag` is active and the sentiment score is strongly positive or negative could be flagged for enhanced monitoring or reduced position sizing, acknowledging that sentiment-driven volatility may exceed model predictions.

Future Work

The results of this project suggest several directions for future research and development:

1. **Transformer-based architectures:** Attention-based sequence models, such as the Temporal Fusion Transformer (TFT), may better capture complex long-range dependencies and the non-uniform relevance of historical time steps compared to standard RNN-family models.
2. **Richer sentiment integration:** The current study uses a single lagged sentiment score. Future work could explore sentiment disaggregation by source type, entity-specific sentiment extraction, and the use of sentiment embeddings from large language models as input features.
3. **Conformal prediction intervals:** The quantile regression approach assumes that the training distribution is representative of the test distribution. Conformal prediction methods offer distribution-free guarantees of coverage validity, which may be more appropriate for the potentially non-stationary financial domain.
4. **Multi-horizon forecasting:** The current models predict one step ahead. Extending to multi-step or multi-horizon forecasting would be directly relevant to portfolio rebalancing and option pricing applications.
5. **Ensemble and hybrid models:** Combining the complementary strengths of statistical models (well-calibrated uncertainty for stationary series) and deep learning models (non-linear pattern recognition) through stacked ensembles may yield improvements over any single model class.
6. **Alternative assets and cross-asset features:** Extending the framework to additional equities or incorporating cross-asset information (e.g. sector indices, macroeconomic indicators, implied volatility measures such as VIX) may improve forecast quality, particularly during regime changes.
7. **Online learning:** Financial time series exhibit regime shifts and structural breaks. Adaptive or online learning frameworks that continuously update model parameters as new data arrives would be more robust to non-stationarity than the static train–test framework employed here.

References

- Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press. Available at: <https://d2l.ai/>
- Nielsen, A. (2019). *Practical Time Series Analysis: Prediction with Statistics and Machine Learning*. O'Reilly Media.